

Assignment 03

Bayesian models & data analysis

1 Estimating the posterior distribution using different computational methods

In a visual word recognition experiment, a participant has to recognize whether a string shown on the screen is a meaningful word (e.g., “book”) or a non-word (e.g., “bktr”). The participant is asked to answer “yes” if the shown string is a meaningful word, and “no” if it is a meaningless non-word. Suppose a participant is shown total $2n$, out of which, n are high frequency words and the other n are low frequency words. These $2n$ words are shown on the screen one by one and you record the recognition time for each word.

In the dataset, `word-recognition.csv`, the column frequency represents whether a word is high frequency word (encoded as 1) or a low frequency word (encoded as 0).

We are interested in testing the following Lexical-access hypothesis.

Lexical-access hypothesis: High frequency words are recognized faster than low frequency words.

We will test whether the data support the above hypothesis or not using a statistical model(s). We can express the lexical-access assumption using the following model.

The lexical-access model

Likelihood assumption:

The recognition time for the i^{th} word, rt_i comes from a finite mixture of lognormal distributions, such that on the i^{th} trial, if the participant is engaged in the task, the rt_i comes from $Lognormal(\mu_i, \sigma)$; But if on the i^{th} trial, the participant is not engaged in the task and is just making random guess, the rt_i comes from $Lognormal(\gamma, \sigma)$.

$$rt_i \sim \begin{cases} Lognormal(\mu_i, \sigma) & \text{if } Z_i = 1 \\ Lognormal(\gamma, \sigma) & \text{if } Z_i = 0 \end{cases} \text{ where } Z_i \sim Bernoulli(\theta)$$

where Z_i indicates whether the participant engages in the task or not on the i^{th} trial; θ is the probability with which the participant engages in the task. A high value of θ would imply that the participant is attentive/sincere, and properly doing the task. The parameter μ_i depends on the frequency of the word i presented in the i^{th} trial, such that

$$\mu_i = \alpha + \beta \cdot \text{frequency}_i$$

The above statements describe your assumptions the underlying generative process. But how would you write the likelihood function for this model. The likelihood function is not straightforward anymore. It is a mixture of two lognormal distributions that occur with some probability. The likelihood function can be given by:

$$\mathcal{L}(\theta, \alpha, \beta, \gamma, \sigma | rt_i) = \theta \cdot \text{dlnorm}(rt_i, \alpha + \beta \text{ frequency}_i, \sigma) + (1 - \theta) \cdot \text{dlnorm}(rt_i, \gamma, \sigma) \quad (1)$$

Prior assumptions:

The model has five parameters, σ , α , β , γ , and θ .

$$\sigma = 0.4$$

$$\alpha = 6$$

$$\beta \sim \text{Normal}(0, 0.5)$$

$$\gamma \sim \text{Normal}(5, 0.5)$$

$$\theta \sim \text{Beta}(70, 30)$$

If the data are consistent with our hypothesis's predictions, the estimated posterior for the β parameter should be negative.¹

Data

The file *word-recognition.csv* contains the recognition times data

```
dat <- read.table("word-recognition.csv", sep=",", header = T)[, -1]
head(dat)
```

```
##   word_id frequency      rt
## 1      1         0 279.0931
## 2      2         1 519.7577
## 3      3         0 231.1684
## 4      4         1 120.2581
## 5      5         0 638.4853
## 6      6         1 603.0779
```

Use the above information to solve the following exercises.

¹However, the parameter estimate does not give any evidence for the hypothesis, it is just tells us about true values of the parameter given the data and assumptions. To evaluate the evidence or to test the hypothesis, we must compare this model against a competing model. In Bayesian framework, hypothesis testing always requires model comparison.

Exercises

1. Use importance sampling to draw samples from the posterior distribution of β , γ , and θ .

(Hint: Draw N samples for each free parameter from some reasonable proposal densities, compute the likelihood and prior for each sample, and store the samples and their corresponding weights as $\{\text{likelihood} \times \text{prior} / \text{proposal density}\}$ in a dataframe/vectors. Now, select $N/2$ samples from the vector of N samples based on their weights. These new $N/2$ samples are the samples from the posterior.

Suppose **proposals** is the vector containing N samples from the proposal density and **weights** is the vector containing their respective likelihood*prior/proposal_density values. You can select $N/2$ samples using

```
post_samples <- sample(proposals,size=N/2,prob=weights)
```

2. Use Markov chain Monte Carlo method to estimate the posterior distribution of β , γ , and θ .
3. Graphically compare the posterior distributions of β obtained using:
 - Grid approximation
 - Markov chain Monte Carlo
4. Does the data support the lexical-access theory's prediction?
(Hint: Check whether the estimated values of β parameter are positive.)
5. What does the estimate of θ parameter tell you?

2 Hamiltonian Monte Carlo

You are given 500 data points that are assumed to come from a normal distribution with mean μ and variance σ^2 ,

```
true_mu <- 800
true_var <- 100 #sigma^2
y <- rnorm(500,mean=true_mu,sd=sqrt(true_var))
hist(y)
```

Copy and paste the above chunk of code to generate data y .

Model:

Let y_i be i^{th} data point,

$$y_i \sim \text{Normal}(\mu, \sigma^2)$$

$$\mu \sim \text{Normal}(m = 1000, s = 100)$$

$$\sigma \sim \text{Normal}(a = 10, b = 2)$$

You have to estimate the posterior distributions of μ and σ using Hamiltonian Monte Carlo sampler.

You can directly use the following HMC sampler code to estimate the parameters of the above model.

```
#Gradient functions
gradient <- function(mu,sigma,y,n,m,s,a,b){
  grad_mu <- (((n*mu)-sum(y))/(sigma^2))+((mu-m)/(s^2))
  grad_sigma <- (n/sigma)-(sum((y-mu)^2)/(sigma^3))+((sigma-a)/(b^2))
  return(c(grad_mu,grad_sigma))
}

#Potential energy function
V <- function(mu,sigma,y,n,m,s,a,b){
  nlpd <- -(sum(dnorm(y,mu,sigma,log=T))+dnorm(mu,m,s,log=T)+dnorm(sigma,a,b,log=T))
  nlpd
}

#HMC sampler
HMC <- function(y,n,m,s,a,b,step,L,initial_q,nsamp,nburn){
  mu_chain <- rep(NA,nsamp)
  sigma_chain <- rep(NA,nsamp)
  reject <- 0
  #Initialization of Markov chain
  mu_chain[1] <- initial_q[1]
  sigma_chain[1] <- initial_q[2]
  #Evolution of Markov chain
  i <- 1
  while(i < nsamp){
    q <- c(mu_chain[i],sigma_chain[i])    # Current position of the particle
    p <- rnorm(length(q),0,1)            # Generate random momentum at the current position
    current_q <- q
    current_p <- p
    current_V = V(current_q[1],current_q[2],y,n,m,s,a,b) # Current potential energy
    current_T = sum(current_p^2)/2        # Current kinetic energy
    # Take L leapfrog steps
    for(l in 1:L){
      # Change in momentum in 'step/2' time
      p <- p-((step/2)*gradient(q[1],q[2],y,n,m,s,a,b))
      # Change in position in 'step' time
      q <- q + step*p
      # Change in momentum in 'step/2' time
      p <- p-((step/2)*gradient(q[1],q[2],y,n,m,s,a,b))
    }
    proposed_q <- q
    proposed_p <- p
    proposed_V = V(proposed_q[1],proposed_q[2],y,n,m,s,a,b) # Proposed potential energy
    proposed_T = sum(proposed_p^2)/2 # Proposed kinetic energy
    accept_prob <- min(1,exp(current_V+current_T-proposed_V-proposed_T))
    # Accept/reject the proposed position q
```

```

    if(accept.prob>runif(1,0,1)){
      mu_chain[i+1] <- proposed_q[1]
      sigma_chain[i+1] <- proposed_q[2]
      i <- i+1
    }else{
      reject <- reject+1
    }
  }
}
posterior <- data.frame(mu_chain,sigma_chain)[-(1:nburn),]
posterior$sample_id <- 1:nrow(posterior)
posterior
}

```

Exercise 2.1 Use the following values for the internal parameters of the HMC sampler,

- Total number of samples, $nsamp = 6000$
- Total number of burn-in samples, $nburn = 2000$ (a certain number of initial samples that you want to throw away)
- Step-size parameter, $step = 0.02$
- Number of leapfrog steps, $L = 12$
- Initializing value of the μ and σ chain, $initial_q = c(1000, 11)$

Estimate and plot the posteriors for μ and σ .

(Hint: You can directly run the HMC function as follows:)

```

df.posterior <- HMC(y=y,n=length(y),           # data
                    m=1000,s=20,a=10,b=2,       # priors
                    step=0.02,                  # step-size
                    L=12,                       # no. of leapfrog steps
                    initial_q=c(1000,11),       # Chain initialization
                    nsamp=6000,                 # total number of samples
                    nburn=2000)                 # number of burn-in samples

```

Exercise 2.2 Check posterior sensitivity to the total number of samples. How do the posteriors change with change in total number of samples?

Estimate and compare the posteriors obtained for the following values of total number of samples, $nsamp$ -

- $nsamp = 100$
- $nsamp = 1000$
- $nsamp = 6000$

Change burn-in samples $nburn$ accordingly. For example, you can set $nburn = nsamp/3$. Keep all other parameters same as in Exercise 1.1.

Exercise 2.3 How do the posteriors change with change in step-size parameter?

Estimate and compare the posteriors obtained when step-size had the following values -

- $step = 0.001$
- $step = 0.005$
- $step = 0.02$

Keep all other parameter same as in Exercise 1.1.

Exercise 2.4 Visually inspect the μ and σ chains obtained in exercise 1.3. Do you find anything problematic?

Exercise 2.5 Check the prior sensitivity for the μ parameter. Estimate and compare the posterior distribution of μ when the prior on μ are -

- $\mu \sim Normal(m = 400, s = 5)$
- $\mu \sim Normal(m = 400, s = 20)$
- $\mu \sim Normal(m = 1000, s = 5)$
- $\mu \sim Normal(m = 1000, s = 20)$
- $\mu \sim Normal(m = 1000, s = 100)$

(Keep all other parameters same as in Exercise 1.1).